

Practical Exam

Name: Sanika Ashok Bhosale

Subject: MERN stack development

Roll No : MC2526012

Class & Div: MCA & 'A'

Q1. create table product table product with name,category,price,stock,brand,availability

- 1. Insert multiple records**
- 2. find all product**
- 3. find product whose value is 4500**
- 4. find the product from Apple brand**
- 5.find the product those area available**
- 6. update the stock of product whose product name="Samsung"**
- 7. delete product by name apple**
- 8. delete all product with stock zero**
- 9. get only product name and price**
- 10. Edit the product whos value is >=500 and set value=350 and dislay app edited product**

Code:

```
const mongoose = require("mongoose");

async function main() {
  try {
    await mongoose.connect("mongodb://127.0.0.1:27017/shopDB");
    console.log("MongoDB Connected");
    const productSchema = new mongoose.Schema({
      name: String,
      category: String,
      price: Number,
      stock: Number,
      brand: String,
      available: Boolean
    });
    const Product = mongoose.model("Product", productSchema);
```

```
await Product.insertMany([
  { name: "iPhone 14", category: "Mobile", price: 70000, stock: 12, brand: "Apple",
    available: true },
  { name: "Samsung M21", category: "Mobile", price: 15000, stock: 8, brand: "Samsung",
    available: true },
  { name: "Boat Earbuds", category: "Audio", price: 2000, stock: 0, brand: "Boat",
    available: false },
  { name: "Apple Watch", category: "Watch", price: 50000, stock: 5, brand: "Apple",
    available: true }
]);

console.log("1. Multiple Records Inserted\n");

console.log("2. All Products:");
console.log(await Product.find());

console.log("\n3. Products with price = 4500:");
console.log(await Product.find({ price: 4500 }));

console.log("\n4. Apple Brand Products:");
console.log(await Product.find({ brand: "Apple" }));

console.log("\n5. Available Products:");
console.log(await Product.find({ available: true }));

await Product.updateMany(
  { name: "Samsung" },
  { $set: { stock: 20 } }
);

console.log("\n6. Samsung Stock Updated to 20");

await Product.deleteMany({ brand: "Apple" });
console.log("\n7. Deleted All Apple Products");
```

```

await Product.deleteMany({ stock: 0 });

console.log("\n8. Deleted Products with Stock 0");


console.log("\n9. Name & Price Only:");
console.log(await Product.find({}, { name: 1, price: 1, _id: 0 }));


await Product.updateMany(
  { price: { $gte: 500 } },
  { $set: { price: 350 } }
);

console.log("\n10. Updated price >=500 to 350");


console.log("\nAll Edited Products:");
console.log(await Product.find());


} catch (err) {
  console.error(err);
} finally {
  mongoose.connection.close();
}
}

main();

```

Output:

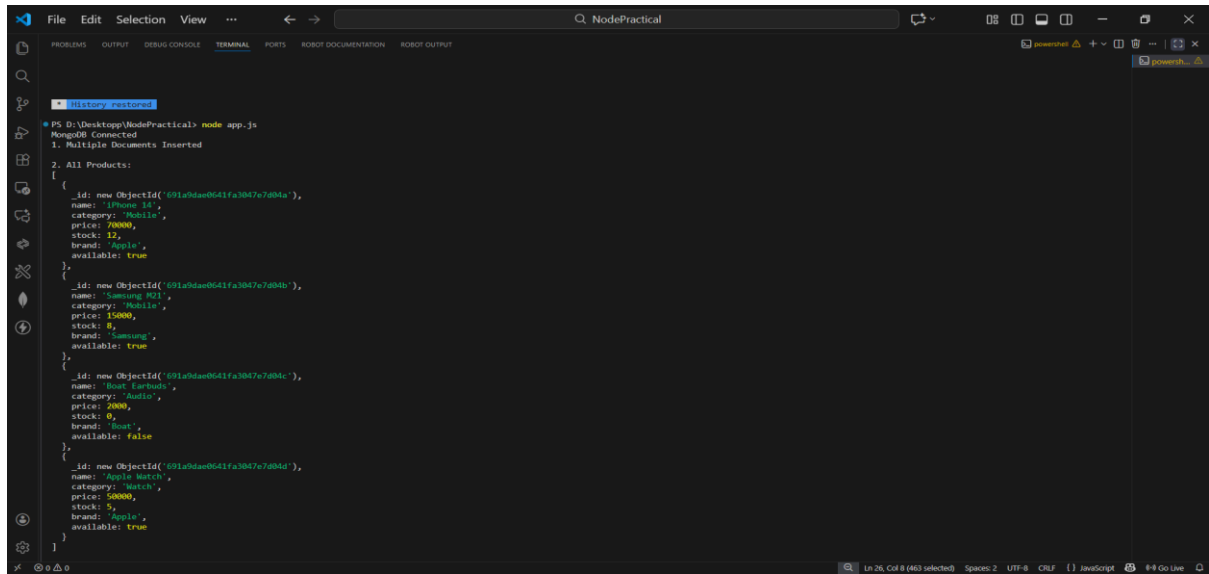
1. Insert multiple records

```

Product.insertMany([
  { name: "iPhone 14", category: "Mobile", price: 70000, stock: 12, brand: "Apple",
    available: true },
  { name: "Samsung M21", category: "Mobile", price: 15000, stock: 8, brand:
    "Samsung", available: true },
  { name: "Boat Earbuds", category: "Audio", price: 2000, stock: 0, brand: "Boat",
    available: false },
  { name: "Apple Watch", category: "Watch", price: 50000, stock: 5, brand:
    "Apple", available: true }
]);

```

2. find all product



```
PS D:\Desktop\NodePractical> node app.js
MongoDB Connected
1. Multiple Documents Inserted

2. All Products:
[
  {
    _id: new ObjectId('691a9dae0641fa3047e7d04a'),
    name: 'iPhone 14',
    category: 'Mobile',
    price: 70000,
    stock: 12,
    brand: 'Apple',
    available: true
  },
  {
    _id: new ObjectId('691a9dae0641fa3047e7d04b'),
    name: 'Samsung S21',
    category: 'Mobile',
    price: 15000,
    stock: 8,
    brand: 'Samsung',
    available: true
  },
  {
    _id: new ObjectId('691a9dae0641fa3047e7d04c'),
    name: 'Bose Earbuds',
    category: 'Audio',
    price: 20000,
    stock: 0,
    brand: 'Bose',
    available: false
  },
  {
    _id: new ObjectId('691a9dae0641fa3047e7d04d'),
    name: 'Apple Watch',
    category: 'Watch',
    price: 50000,
    stock: 5,
    brand: 'Apple',
    available: true
  }
]
```

3. Product with price = 5000:
[]

4. Products from brand Apple:

```
[
  {
    _id: new ObjectId('691a9dae0641fa3047e7d04a'),
    name: 'iPhone 14',
    category: 'Mobile',
    price: 70000,
    stock: 12,
    brand: 'Apple',
    available: true
  },
  {
    _id: new ObjectId('691a9dae0641fa3047e7d04d'),
    name: 'Apple Watch',
    category: 'Watch',
    price: 50000,
    stock: 5,
    brand: 'Apple',
    available: true
  }
]
```

```

5. Available Products:
[
  {
    _id: new ObjectId('691a9dae0641fa3047e7d04a'),
    name: 'iPhone 14',
    category: 'Mobile',
    price: 70000,
    stock: 12,
    brand: 'Apple',
    available: true
  },
  {
    _id: new ObjectId('691a9dae0641fa3047e7d04b'),
    name: 'Samsung M21',
    category: 'Mobile',
    price: 15000,
    stock: 8,
    brand: 'Samsung',
    available: true
  },
  {
    _id: new ObjectId('691a9dae0641fa3047e7d04d'),
    name: 'Apple Watch',
    category: 'Watch',
    price: 50000,
    stock: 5,
    brand: 'Apple',
    available: true
  }
]

```

```

6. Samsung product stock updated to 20

```

```

7. Deleted all products with brand Apple

```

```

8. Deleted products with stock = 0

```

```

9. Name & Price Only:
[
  { name: 'Samsung M21', price: 15000 },
  { name: 'Samsung M21', price: 15000 },
  { name: 'Samsung M21', price: 15000 }
]

```

```

10. Updated price >=500 to 350

All Edited Products:
[
  {
    _id: new ObjectId('691a9dae0641fa3047e7d04b'),
    name: 'Samsung M21',
    category: 'Mobile',
    price: 350,
    stock: 20,
    brand: 'Samsung',
    available: true
  },
  {
    _id: new ObjectId('691a9f130839371231ae2f19'),
    name: 'Samsung M21',
    category: 'Mobile',
    price: 350,
    stock: 20,
    brand: 'Samsung',
    available: true
  },
  {
    _id: new ObjectId('691aa0409c7785b105f1b1b4'),
    name: 'Samsung M21',
    category: 'Mobile',
    price: 350,
    stock: 8,
    brand: 'Samsung',
    available: true,
    __v: 0
  }
]

```

Q2. Create a text file and read its content using Aync/Await with the FS module in node.js.

Code:

```
const fs = require('fs/promises');

async function main() {
  try {
    await fs.writeFile('example.txt', 'Hello Sanika! \nWelcome to Node.js!');
    console.log('File created successfully.');
```

const data = await fs.readFile('example.txt', 'utf-8');

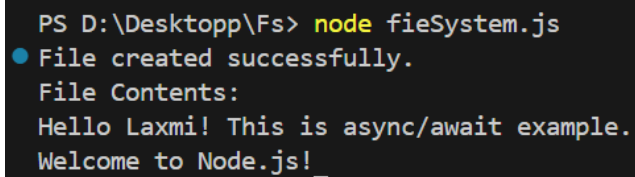
console.log('File Contents:');

console.log(data);

```
  } catch (err) {
    console.error('Error:', err);
  }
}
```

main();

Output:



```
PS D:\Desktop\Fs> node fieSystem.js
● File created successfully.
File Contents:
Hello Laxmi! This is async/await example.
Welcome to Node.js!_
```